

Name: Nagamalla Sai Ganesh

Hands on 1

Create a Spring Web Project using Maven

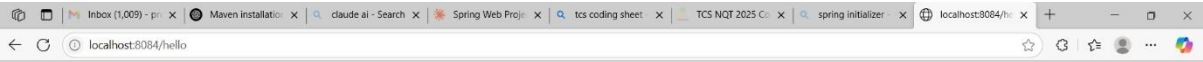
```
Command Prompt
Downloading from central: https://repo.maven.apache.org/maven2/org/tomlj/tomlj/1.0.0/tomlj-1.0.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5-h2/5.3.4/httpcore5-h2-5.3.4.jar (242 kB at 169 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/antlr/antlr4-runtime/4.7.2/antlr4-runtime-4.7.2.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/tomlj/tomlj/1.0.0/tomlj-1.0.0.jar (157 kB at 106 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/google/code/findbugs/jsr305/3.0.2/jsr305-3.0.2.jar
Downloading from central: https://repo.maven.apache.org/maven2/com/google/code/findbugs/jsr305/3.0.2/jsr305-3.0.2.jar (20 kB at 13 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-loader-tools/3.5.3/spring-boot-loader-tools-3.5.3.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/antlr/antlr4-runtime/4.7.2/antlr4-runtime-4.7.2.jar (338 kB at 183 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/micrometer/micrometer-observation/1.14.8/micrometer-observation-1.14.8.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/client5/httpclient5/5.5/httpclient5-5.5.jar (955 kB at 491 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/micrometer/micrometer-commons/1.14.8/micrometer-commons-1.14.8.jar
Downloading from central: https://repo.maven.apache.org/maven2/io/micrometer/micrometer-observation/1.14.8/micrometer-observation-1.14.8.jar (75 kB at 38 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-shade-plugin/3.6.0/maven-shade-plugin-3.6.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/io/micrometer/micrometer-commons/1.14.8/micrometer-commons-1.14.8.jar (48 kB at 24 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.7/asm-9.7.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-loader-tools/3.5.3/spring-boot-loader-tools-3.5.3.jar (46
6 kB at 226 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.7/asm-commons-9.7.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-shade-plugin/3.6.0/maven-shade-plugin-3.6.0.jar (150 kB at 70 k
B/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.7/asm-tree-9.7.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.7/asm-9.7.jar (125 kB at 58 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.7/asm-commons-9.7.jar (73 kB at 34 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.10/jdependency-2.10.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.7/asm-tree-9.7.jar (52 kB at 23 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5/5.3.4/httpcore5-5.3.4.jar (909 kB at 402 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/net/java/dev/jna/jna/5.17.0/jna-5.17.0.jar (2.0 MB at 881 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.10/jdependency-2.10.jar (416 kB at 172 kB/s)
[INFO] Replacing main artifact C:\Users\kpre0\Downloads\spring-learn\spring-learn\target\spring-learn-0.0.1-SNAPSHOT.jar with repackaged archive, adding nes
ted dependencies in BOOT-INF.
[INFO] The original artifact has been renamed to C:\Users\kpre0\Downloads\spring-learn\spring-learn\target\spring-learn-0.0.1-SNAPSHOT.jar.original
[INFO] BUILD SUCCESS
[INFO]
```

- spring-learn
 - src/main/java
 - com.cognizant.spring_learn
 - com.cognizant.springlearn
 - HelloController.java
 - SpringLearnApplication.java
 - src/main/resources
 - static
 - templates
 - application.properties
 - src/test/java
 - com.cognizant.spring_learn
 - JRE System Library [JavaSE-21]
 - Maven Dependencies
 - bin
 - src
 - target
 - HELP.md
 - mvnw
 - mvnw.cmd
 - pom.xml

```
librarymanageme ApplicationCont Strong.java SpringLearnAppl application.pro HelloController »19
1 package com.cognizant.springlearn;
2
3 import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 public class SpringLearnApplication {
8
9     private static final Logger logger = LoggerFactory.getLogger(SpringLearnApplication.class);
10
11     public static void main(String[] args) {
12         logger.info("Starting SpringLearnApplication...");
13         SpringApplication.run(SpringLearnApplication.class, args);
14         logger.info("SpringLearnApplication started successfully!");
15     }
16 }
17
18
19 }
```

```
librarymanageme ApplicationCont Strong.java SpringLearnAppl application.pro HelloController »19
1 package com.cognizant.springlearn;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class HelloController {
8
9     @GetMapping("/hello")
10     public String hello() {
11         return "Hello from Spring Boot!";
12     }
13 }
14 }
```

OUTPUT



Spring Core – Load Country from Spring Configuration XML

- ▼  spring-learn
 - ▼  src/main/java
 -  com.cognizant.spring_learn
 - ▼  com.cognizant.springlearn
 - >  Country.java
 - >  HelloController.java
 - >  SpringLearnApplication.java
 - ▼  src/main/resources
 -  static
 -  templates
 -  application.properties
 -  country.xml
 - >  src/test/java
 - >  JRE System Library [JavaSE-21]
 - >  Maven Dependencies
 - ▼  bin
 - >  src
 - >  target
 -  HELP.md
 -  mvnw
 -  mvnw.cmd
 -  pom.xml

```
librarymanagemer ApplicationCont SpringLearnAppl HelloController country.xml Country.java 21
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans
5     http://www.springframework.org/schema/beans/spring-beans.xsd">
6
7     <bean id="country" class="com.cognizant.springlearn.Country">
8         <property name="code" value="IN"/>
9         <property name="name" value="India"/>
10    </bean>
11
12 </beans>
13
```

```
Eclipse - spring-learn/src/main/java/com/cognizant/springlearn/Country.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
1 package com.cognizant.springlearn;
2
3 import org.slf4j.Logger;
4
5 public class Country {
6     private static final Logger LOGGER = LoggerFactory.getLogger(Country.class);
7
8     private String code;
9     private String name;
10
11     public Country() {
12         LOGGER.debug("inside Country Constructor.");
13     }
14
15     public String getCode() {
16         LOGGER.debug("inside getCode");
17         return code;
18     }
19
20     public void setCode(String code) {
21         LOGGER.debug("inside setCode");
22         this.code = code;
23     }
24
25     public String getName() {
26         LOGGER.debug("inside getName");
27         return name;
28     }
29
30     public void setName(String name) {
31         LOGGER.debug("inside setName");
32         this.name = name;
33     }
34
35     @Override
36     public String toString() {
37         return "Country{" +
38             "code='" + code + '\'' +
39             ", name='" + name + '\'' +
40             '}';
41     }
42
43 }
44
```

OUTPUT

```
Problems Javadoc Declaration Console Coverage
<terminated> SpringLearnApplication [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul
Country : Country{code='IN', name='India'}
```

Hello World RESTful Web Service

- ▼  spring-learn
 - ▼  src/main/java
 -  com.cognizant.spring_learn
 - ▼  com.cognizant.springlearn
 - >  Country.java
 - >  HelloController.java
 - >  SpringLearnApplication.java
 - ▼  src/main/resources
 -  static
 -  templates
 -  application.properties
 -  country.xml
 - >  src/test/java
 - >  JRE System Library [JavaSE-21]
 - >  Maven Dependencies
 - ▼  bin
 - >  src
 - >  target
 -  HELP.md
 -  mvnw
 -  mvnw.cmd
 -  pom.xml
 - ▼  src
 -  main
 -  test
 - >  target
 -  HELP.md
 -  mvnw
 -  mvnw.cmd
 -  pom.xml

```
librarymanageme | SpringLearnAppl | HelloController | country.xml | Country.java | spring-learn/bi | »22
1 package com.cognizant.springlearn;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7
8 @SpringBootApplication
9 public class SpringLearnApplication {
10
11     private static final Logger LOGGER = LoggerFactory.getLogger(SpringLearnApplication.class);
12
13     public static void main(String[] args) {
14         LOGGER.info("START of main()");
15         SpringApplication.run(SpringLearnApplication.class, args);
16         LOGGER.info("END of main()");
17     }
18 }
19
```

```
librarymanageme | SpringLearnAppl | HelloController | country.xml | Country.java | spring-learn/bi | »22
1 package com.cognizant.springlearn;
2 import org.springframework.web.bind.annotation.RestController;
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.web.bind.annotation.GetMapping;
6
7 @RestController
8 public class HelloController {
9
10     private static final Logger LOGGER = LoggerFactory.getLogger(HelloController.class);
11
12     @GetMapping("/hello")
13     public String sayHello() {
14         LOGGER.info("START sayHello");
15         String response = "Hello World!!";
16         LOGGER.info("END sayHello");
17         return response;
18     }
19 }
```

OUTPUT

Overview

GET http://localhost:8084/hello

No environment

http://localhost:8084/hello

Save

Share

GET

http://localhost:8084/hello

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (5)

Test Results

200 OK • 25 ms • 177 B •

Raw

Preview

Visualize

1

Hello World!!

localhost:8084/hello

Hello World!!

REST - Country Web Service

```
CountryControll  country.xml  Country.java  spring-learn/bi  SpringLearnAppl  SpringLearnAppl  »24
1 package com.cognizant.springlearn;
2 import org.springframework.boot.SpringApplication;
3 import org.springframework.boot.autoconfigure.SpringBootApplication;
4 @SpringBootApplication
5 public class SpringLearnApplication {
6     public static void main(String[] args) {
7         SpringApplication.run(SpringLearnApplication.class, args);
8     }
9 }
10
11
12
```

```
CountryControll  country.xml  Country.java  spring-learn/bi  SpringLearnAppl  SpringLearnAppl  »24
1 package com.cognizant.springlearn;
2
3 import org.slf4j.Logger;
4
5 @RestController
6 public class CountryController {
7
8     private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
9
10    @RequestMapping("/country")
11    public Country getCountryIndia() {
12        LOGGER.info("START getCountryIndia");
13
14        ClassPathXmlApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
15        Country country = context.getBean("country", Country.class);
16        context.close();
17
18        LOGGER.info("END getCountryIndia");
19        return country;
20    }
21 }
22
23
```

OUTPUT

HTTP

http://localhost:8084/country

Save

Share

GET

http://localhost:8084/country

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (5)

Test Results

200 OK

42 ms

192 B

...

{ } JSON

Preview

Visualization

1 {

2 "code": "IN",

3 "name": "India"

4 }

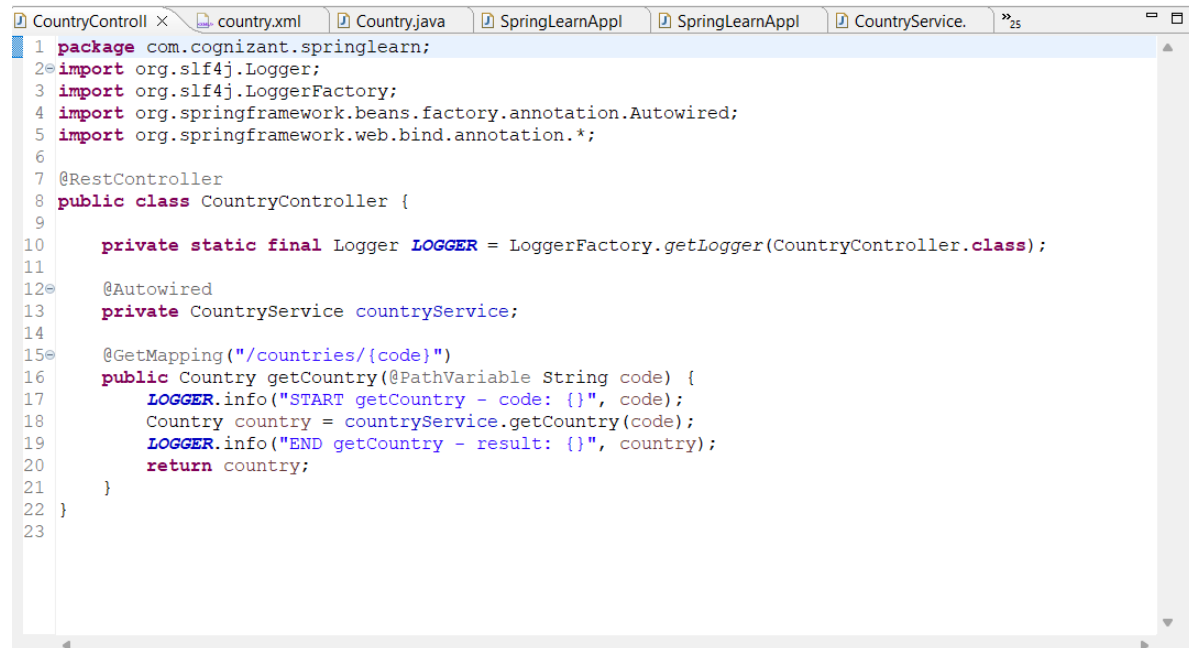
localhost:8084/country

...

Pretty-print

{ "code": "IN", "name": "India" }

REST - Get country based on country code



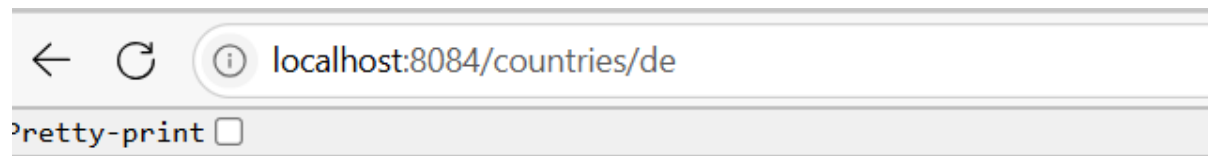
```
1 package com.cognizant.springlearn;
2 import org.slf4j.Logger;
3 import org.slf4j.LoggerFactory;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.web.bind.annotation.*;
6
7 @RestController
8 public class CountryController {
9
10     private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
11
12     @Autowired
13     private CountryService countryService;
14
15     @GetMapping("/countries/{code}")
16     public Country getCountry(@PathVariable String code) {
17         LOGGER.info("START getCountry - code: {}", code);
18         Country country = countryService.getCountry(code);
19         LOGGER.info("END getCountry - result: {}", country);
20         return country;
21     }
22 }
23
```

```
CountryControll | country.xml | Country.java | SpringLearnAppl | SpringLearnAppl | CountryService. X 25
1 package com.cognizant.springlearn;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5
6 import java.util.List;
7
8 @Service
9 public class CountryService {
10
11     public Country getCountry(String code) {
12         ClassPathXmlApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
13
14         @SuppressWarnings("unchecked")
15         List<Country> countryList = (List<Country>) context.getBean("countryList");
16
17         context.close();
18
19         return countryList.stream()
20             .filter(c -> c.getCode().equalsIgnoreCase(code))
21             .findFirst()
22             .orElse(null); // Or throw error if not found
23     }
24 }
25
```

OUTPUT

```
localhost:8084/countries/in
Pretty-print
{"code":"IN","name":"India"}
```

```
localhost:8084/countries/jp
Click to go back, hold to see history
{"code":"JP","name":"Japan"}
```



```
{"code": "DE", "name": "Germany"}
```

Create authentication service that returns JWT

```
CountryControll | Country.java | SpringLearnAppl | CountryService. | spring-learn/po | *Authentication x | »27
1 package com.cognizant.springlearn;
2 import io.jsonwebtoken.Jwts;
3 import io.jsonwebtoken.SignatureAlgorithm;
4 import io.jsonwebtoken.security.Keys;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.*;
7 import javax.crypto.SecretKey;
8 import java.util.Base64;
9 import java.util.Date;
10
11 @RestController
12 public class AuthenticationController {
13
14     private static final SecretKey key = Keys.secretKeyFor(SignatureAlgorithm.HS256);
15
16     @GetMapping("/authenticate")
17     public ResponseEntity<?> authenticate(@RequestHeader("Authorization") String authHeader) {
18         if (authHeader == null || !authHeader.startsWith("Basic ")) {
19             return ResponseEntity.status(401).body("Missing or invalid Authorization header");
20         }
21
22         // Decode Base64 encoded credentials
23         String base64Credentials = authHeader.substring("Basic ".length());
24         byte[] credDecoded = Base64.getDecoder().decode(base64Credentials);
25         String credentials = new String(credDecoded);
26         String[] values = credentials.split(":", 2);
27     }
28 }
```

OUTPUT

```
Command Prompt
Microsoft Windows [Version 10.0.26100.4652]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kpre0>curl -s -u user:pwd http://localhost:8084/authenticate
{"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyIiwiaWF0IjoxNzUyMzkxMTYzLCJleHAiOi0jE3NTIzOTA3NjN9.YBEv1IJKkqbwCj7j4AHqPH2b
r9LH5aAjjC-1RCUGEmc"}
C:\Users\kpre0>
```